Ella Fortenbery
COMP30230 Final Project
5 December 2025

Connectionist Computing

Multilayer Perceptrons (MLP) are connectionist networks that model more than one layer of neural units. There is a hidden layer of units that allows for the learning to occur. Each node is connected to each node in the next layer by different weights. There is a forward computation to find the currently predicted output, and then the error is evaluated and the weights are updated to better fit the data by backpropagation.

The first choice I had to implement was how to initialize the weights. Weights should always be small random values, but there are different methods for starting them. Xavier Weight Initialization is widely used to avoid vanishing or exploding gradients. It works well with both the upper and lower layers' activation functions. I chose to use *tanh* as the activation function for the hidden layer's weights because *tanh* prevents vanishing gradients and is meant for hidden inputs. Its output is symmetric and zero-centered, and its derivative is computationally simple. I also decided to add bias units into my hidden layer and output layer. These allow the model to better fit the data by providing a constant weight.

$$w = random[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}]$$

Formula 1 - Xavier Weight Initialization: random uniform distribution where n is number of units in the previous layer

$$h_j = tanh[\Sigma(x_i * w_{ij}) + b_j]$$

Formula 2 - Forward calculation for each hidden unit. x is the inputs, w is the weights, b is the bias, and h is the hidden unit.

$$y_k = f[\Sigma(h_j * w_{jk}) + b_k]$$

Formula 3 - Forward calculation for each output unit. y is the output.

$$\Delta w = p(y - t)$$

Formula 4 - Backward calculation weight update where p is the previous layer and t is the target output.

The first dataset I used to test my model was to model the XOR function, so it has 2 binary inputs and 1 binary output. I used 4 hidden units. After training, my model was able to accurately predict the correct output for all 4 input combinations. The activation function used on the output layer was sigmoidal because it needed to have a binary output. There were some issues with having very slow learning because of the small number of examples, but when the learning rate was high enough it converged very quickly: less than 200 epochs. Higher learning rates have a faster convergence on a low error.

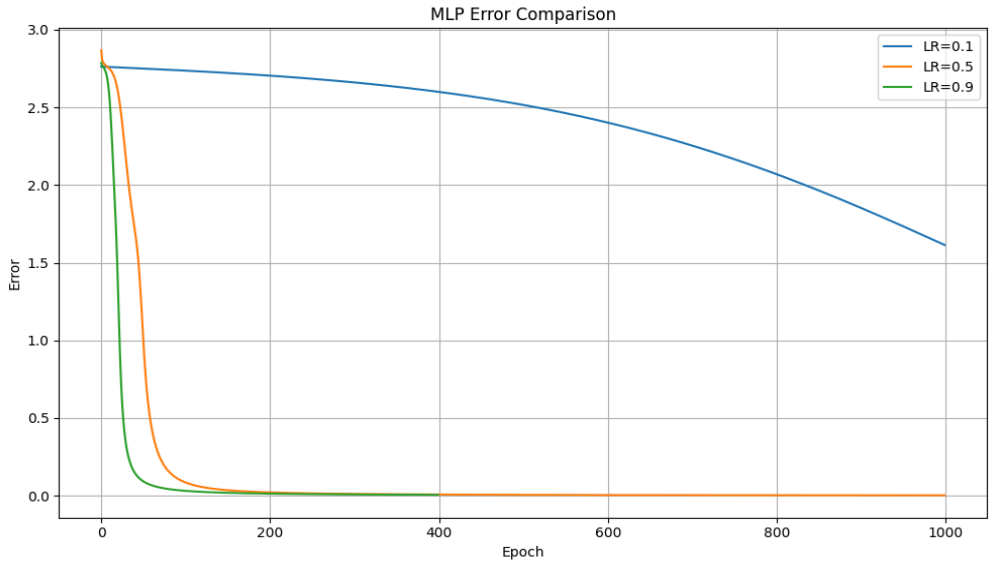


Figure 1 - Every epoch, the error in the model's prediction decreases.

The next data set to train my model on was a sine combination function. The four randomly generated inputs were combined and put through a sine function to make one real-numbered output. I used a linear activation function for the output layer because the results needed to be real numbers. I trained the model on 8 hidden units.

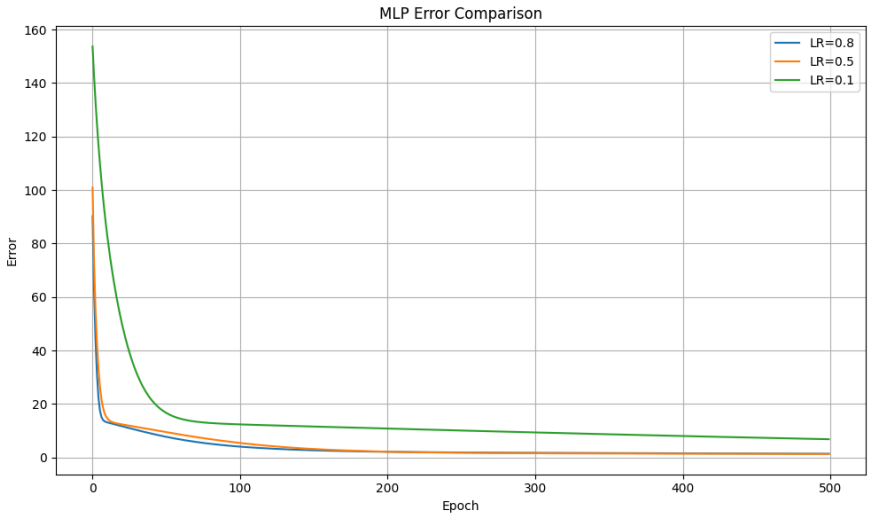
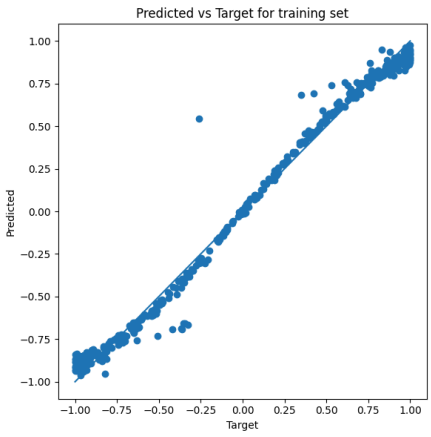
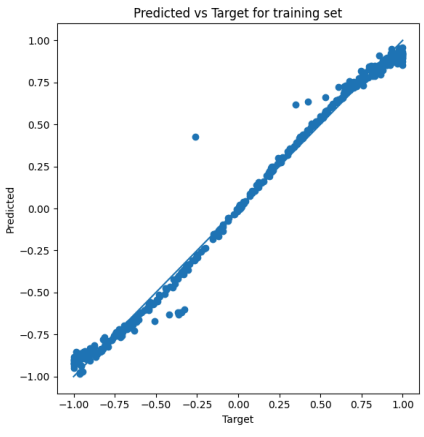


Figure 2 - Error at each epoch for different learning rates.

This error graph shows that both learning rates of 0.5 and 0.8 have very similar errors after training, but this is only on the training set. Below, I further analyzed how well my model predicted the training set and the evaluation set. I plotted the target against the predicted sine output. To calculate the error of the model, I found the mean squared error between each coordinate pair and the line with slope=1.

Learning Rate	0.5	0.8
Training Set	 0.0068	 0.0048

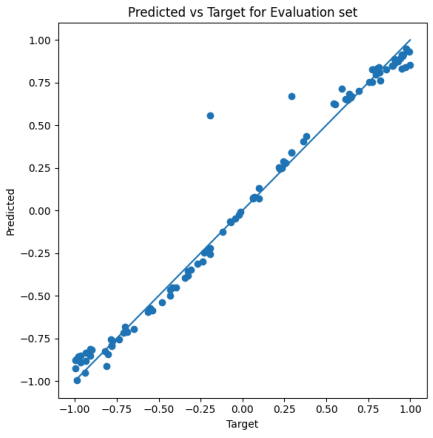
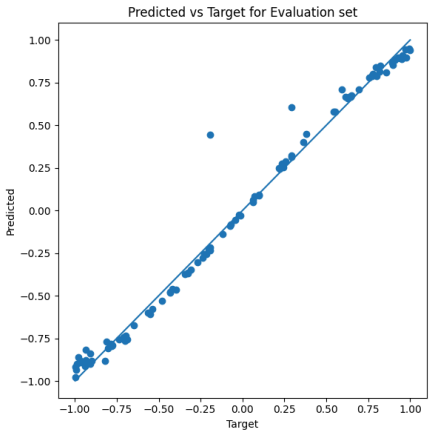
Learning Rate	0.5	0.8
Evaluation Set	 0.0099	 0.0070

Figure 3 - Mean error squared for each set at different learning rates. The predicted output is plotted against the target values.

The final test for my model was letter recognition based on 16 different features (the inputs). I encoded each letter to an array with 26 zero indices, with the letter's index as 1.0. Originally, MLP was 0% accurate because it would guess 0 for all 26 outputs using the sigmoidal output. I implemented the softmaxxing function to force the model to pick one letter. I also had to do learning in mini batches because the data set is so large. The error rate then was 36.73%. Next, I increased the number of hidden units from 12 to 64 and I had 10.98% error. I then shuffled the examples in between every epoch which didn't make the accuracy better, but the model did converge faster. The last thing I did to increase the accuracy was normalize the input data with the z-score of the training set. I used those same statistics to normalize the evaluation set to avoid any data leakage. This brought the error on the evaluation set down to 9.38%.

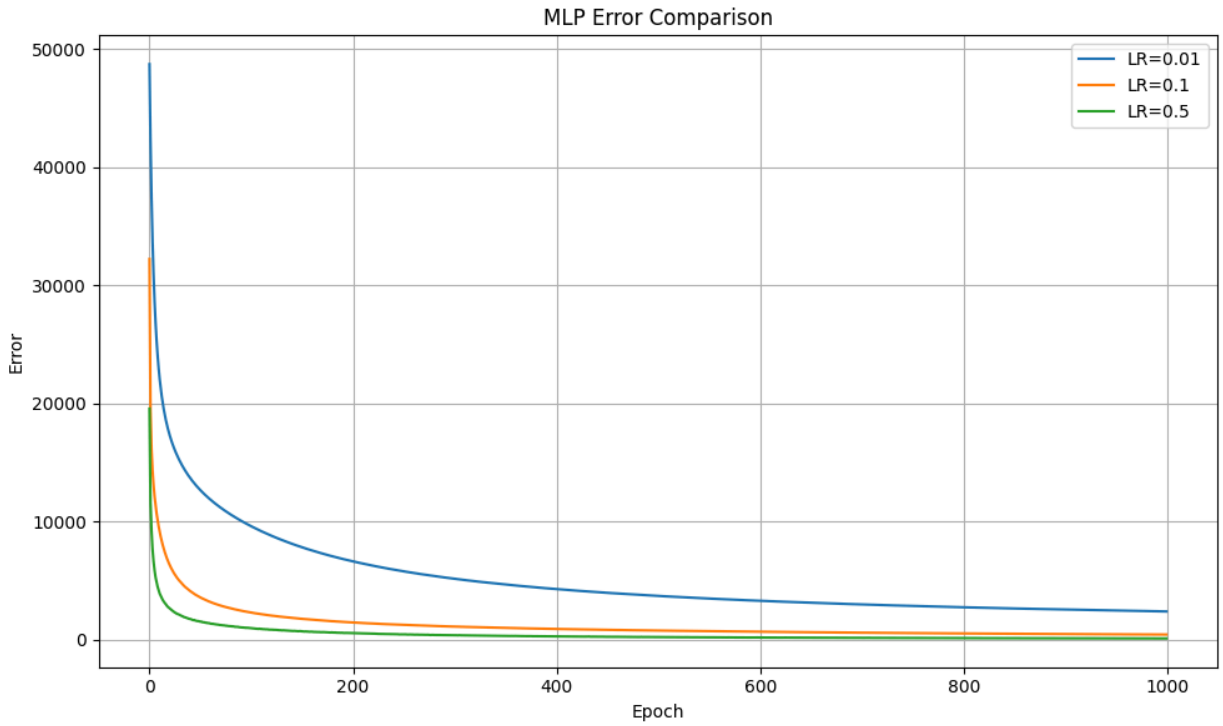


Figure 4 - The error over 1000 epochs for letter recognition

Creating my own learning model taught me about what processes are actually taking place in connectionist computing. All of the formulas and strategies from class worked together to create a very simple program. I learned how to research different options for the process and make choices to better fit the specific test I was solving. I did this to choose the Xavier weight initialization, tanh hidden unit activation function, and to include bias units. Now I understand how hidden units and weights actually contribute to a computer predicting a certain output. The input nodes are multiplied and combined by weight percentages through the tanh function, then again to make an output. The bias units allow for offset in the weight values. I also learned the difference between the sigmoid, tanh, linear, and softmax activation formulas. Overall, I have a much clearer understanding of what a machine learning model does, especially a multilayer perceptron.